Capstone: Data Analysis Report

Capstone D606 Task 2

Duvall Roberts

A. Research Question Financial institutions can utilize predictive modeling to estimate customer attrition and deploy targeted interventions. Utilizing an advanced statistical classification modeling framework is highly effective on classification data as a predictive method (James, Witten, Hastie, & Tibshirani, 2021). The specific context of this study focuses on product categories as a structurally significant attribute in churn-signal detection; baseline data demonstrates that debt collection narratives showed a 0% positive churn rate compared to 5.3-6.2% for checking/savings and credit card complaints, reflecting that debt collection is an assigned relationship rather than a chosen one (Mavice, n.d.). The research question asks: Do consumer complaint product categories have a statistically significant association with customer churn signals in the CFPB Churn Signal Golden Set?

- **H0 (Null Hypothesis):** There is no statistically significant association between complaint product category and churn signal.
- **Ha (Alternative Hypothesis):** There is a statistically significant association between complaint product category and churn signal.
- **Decision Rule:** At a significance level of $\alpha = 0.05$, the null hypothesis is rejected when the p-value is less than 0.05.

B. Data Collection The data collected is the "CFPB Churn Signal Golden Set," a publicly available evaluation dataset containing 1,001 hand-verified consumer complaint narratives. The dataset maps categorical independent variables (complaint ID, product, fold) and unstructured text (narrative) against a binary dependent variable (churn signal). An advantage of this dataset is its high reliability for statistical analysis due to its hand-verified labels (Mavice, n.d.). A disadvantage of this data-gathering methodology is the small initial size and extreme class imbalance. To overcome this challenge, the SMOTE synthetic data generation technique was applied (Bruce, Bruce, & Gedeck, 2020). Because the original dataset contained severe class imbalance, SMOTE was applied across the dataset to address the severe class imbalance and provide a more balanced dataset for the machine learning analysis prior to cross-validation splitting. Using a duplicate size of 135, the initial 1,001-row dataset was expanded to 7,481 rows, shifting the class balance to 953 negative and 6,528 positive churn signals.

C. Data Extraction and Preparation Data was extracted and prepared using a hybrid R and Python pipeline to ensure structural data integrity (Wickham, Çetinkaya-Rundel, & Grolemund, 2023).

- **Step 1: Data Extraction and Merging**

```
===== R/01_load_data.R =====

> # Loads golden_set.csv + cv_folds.csv, merges on complaint_id, and keeps only
> # the true IV/DV columns per the approved Task 1 variable table.
> # .... [TRUNCATED]

> library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
> library(stringr)

> golden <- read_csv("data/golden_set.csv", show_col_types = FALSE)

> folds  <- read_csv("data/cv_folds.csv",  show_col_types = FALSE)

> stopifnot(
+   n_distinct(golden$complaint_id) == nrow(golden),
+   n_distinct(folds$complaint_id)  == nrow(folds)
+ )

> df <- golden %>%
+   inner_join(folds, by = "complaint_id") %>%
+   transmute(
+     complaint_id,
+     product = factor(product,
+                 .... [TRUNCATED]

> stopifnot(nrow(df) == nrow(golden))  # confirms the join dropped nothing

> dir.create("data/processed", showWarnings = FALSE)
```

```
> dir.create("data/processed", showWarnings = FALSE)

> saveRDS(df, "data/processed/model_frame.rds")

> cat("Loaded", nrow(df), "rows.\n")
Loaded 1001 rows.

> print(table(df$product))

Checking or savings account                 Credit card             Debt collection
                        417                         417                         167

> cat("Overall churn rate:", mean(df$churn_signal), "\n")
Overall churn rate: 0.04795205
```

Data was extracted via CSV and merged on the complaint ID field. Missing fields were addressed, and only the required independent and dependent variables were retained for the analysis frame.

- **Step 2: NLP Embeddings via Python Integration**

```
===== R/02_get_embeddings.R =====

> # Bridges to Python via reticulate to compute DistilBERT embeddings +
> # sentiment for every narrative, cached to disk since CPU inference over
> # .... [TRUNCATED]

> if (Sys.getenv("RETICULATE_PYTHON") == "") {
+   reticulate::use_virtualenv("d606-nlp", required = TRUE)
+ }

> cache_path <- "data/processed/embeddings_cache.rds"

> df <- readRDS("data/processed/model_frame.rds")

> if (file.exists(cache_path)) {
+   cat("Loading cached embeddings...\n")
+   cached <- readRDS(cache_path)
+ } else {
+   cat("Computing DistilBERT  ..." ... [TRUNCATED]
Computing DistilBERT embeddings + sentiment (this can take a while)...
C:\Users\USER\DOCUME~1\VIRTUA~1\d606-nlp\Lib\site-packages\huggingface_hub\file_download.py:143: UserWarning: `huggingf
ace_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support
them in C:\Users\USER\.cache\huggingface\hub\models--distilbert-base-uncased. Caching files will still work but in a de
graded version that might require more space on your disk. This warning can be disabled by setting the `HF_HUB_DISABLE_
SYMLINKS_WARNING` environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#
limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In ord
er to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-
device-for-development
  warnings.warn(message)
C:\Users\USER\DOCUME~1\VIRTUA~1\d606-nlp\Lib\site-packages\huggingface_hub\file_download.py:949: FutureWarning: `resume
_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to fo
rce a new download, use `force_download=True`.
  warnings.warn(
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download.
For better performance, install the package with: `pip install huggingface_hub[hf_xet]` or `pip install hf_xet`
C:\Users\USER\DOCUME~1\VIRTUA~1\d606-nlp\Lib\site-packages\huggingface_hub\file_download.py:143: UserWarning: `huggingf
ace_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support
them in C:\Users\USER\.cache\huggingface\hub\models--distilbert-base-uncased-finetuned-sst-2-english. Caching files wil
l still work but in a degraded version that might require more space on your disk. This warning can be disabled by sett
ing the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details, see https://huggingface.co/docs/huggi
ngface_hub/how-to-cache#limitations.
```

```
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In ord
er to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-
device-for-development
  warnings.warn(message)
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download.
For better performance, install the package with: `pip install huggingface_hub[hf_xet]` or `pip install hf_xet`
embedded 16/1001
embedded 32/1001
embedded 48/1001
embedded 64/1001
embedded 80/1001
embedded 96/1001
embedded 112/1001
embedded 128/1001
embedded 144/1001
embedded 160/1001
embedded 176/1001
embedded 192/1001
embedded 208/1001
embedded 224/1001
embedded 240/1001
embedded 256/1001
embedded 272/1001
embedded 288/1001
embedded 304/1001
embedded 320/1001
embedded 336/1001
embedded 352/1001
embedded 368/1001
embedded 384/1001
embedded 400/1001
embedded 416/1001
embedded 432/1001
embedded 448/1001
embedded 464/1001
embedded 480/1001
embedded 496/1001
embedded 512/1001
embedded 528/1001
embedded 544/1001
embedded 560/1001
embedded 576/1001
```

```
embedded 560/1001
embedded 576/1001
embedded 592/1001
embedded 608/1001
embedded 624/1001
embedded 640/1001
embedded 656/1001
embedded 672/1001
embedded 688/1001
embedded 704/1001
embedded 720/1001
embedded 736/1001
embedded 752/1001
embedded 768/1001
embedded 784/1001
embedded 800/1001
embedded 816/1001
embedded 832/1001
embedded 848/1001
embedded 864/1001
embedded 880/1001
embedded 896/1001
embedded 912/1001
embedded 928/1001
embedded 944/1001
embedded 960/1001
embedded 976/1001
embedded 992/1001
embedded 1001/1001

> stopifnot(identical(cached$complaint_id, df$complaint_id))

> cat("Embedding matrix:", nrow(cached$embeddings), "x", ncol(cached$embeddings), "\n")
Embedding matrix: 1001 x 768
```

The reticulate package was utilized to bridge R and Python, generating a 1001 x 768

embedding matrix via Hugging Face DistilBERT to transform the unstructured narratives into numerical features for natural language processing. A Shapiro-Wilk test was then used to check the normality of these continuous embeddings.

- **Step 3: Synthetic Data Expansion**

```
===== R/03_smote_expand.R =====

> # SMOTE on the full 1,001-row golden set, run BEFORE any analysis, per the
> # approved Task 1 order and the rubric's 7,000-row minimum. Needs numer .... [TRUNCATED]

> library(smotefamily)

> df  <- readRDS("data/processed/model_frame.rds")

> emb <- readRDS("data/processed/embeddings_cache.rds")

> embed_df <- as.data.frame(emb$embeddings)

> emb_cols <- names(embed_df)

> full <- df %>%
+   mutate(sentiment_score = emb$sentiment_score[match(complaint_id, emb$complaint_id)]) %>%
+   bind_cols(embed_df)

> product_dummies <- as.data.frame(model.matrix(~ product, data = full))[, -1, drop = FALSE]

> names(product_dummies) <- make.names(names(product_dummies))

> dummy_cols <- names(product_dummies)

> X <- cbind(full[, emb_cols], full[, "sentiment_score", drop = FALSE], product_dummies)

> n_real      <- nrow(full)

> minority_n  <- sum(full$churn_signal == 1)

> target_total <- 7500

> dup_size <- max(1, round((target_total - n_real) / minority_n))  # lands close to ~7,500

> cat("Pre-SMOTE:", n_real, "rows | dup_size =", dup_size, "\n")
Pre-SMOTE: 1001 rows | dup_size = 135

> smote_out <- SMOTE(X = X, target = full$churn_signal, K = 5, dup_size = dup_size)

> sm_data <- smote_out$data
```

```
> names(sm_data)[names(sm_data) == "class"] <- "churn_signal"

> sm_data$churn_signal <- as.integer(as.character(sm_data$churn_signal))

> n_total <- nrow(sm_data)

> sm_data$is_synthetic <- c(rep(FALSE, n_real), rep(TRUE, n_total - n_real))

> sm_data$row_id       <- seq_len(n_total)   # stable id used by every later join

> sm_data$complaint_id <- c(full$complaint_id, rep(NA_real_, n_total - n_real))

> # Reconstruct a categorical `product` for synthetic rows via argmax of the
> # interpolated one-hot dummies -- the only way back to a discrete label .... [TRUNCATED]

> other_levels   <- levels(full$product)[-1]

> reconstruct_product <- function(vals) {
+   if (all(vals < 0.5)) baseline_level else other_levels[which.max(vals)]
+ }

> synthetic_dummies  <- as.matrix(sm_data[(n_real + 1):n_total, dummy_cols])

> synthetic_products <- apply(synthetic_dummies, 1, reconstruct_product)

> sm_data$product <- factor(c(as.character(full$product), synthetic_products),
+                           levels = levels(full$product))

> cat("Post-SMOTE:", n_total, "rows\n")
Post-SMOTE: 7481 rows

> print(table(sm_data$churn_signal))

   0    1
 953 6528

> print(table(sm_data$is_synthetic))

FALSE  TRUE
 1001  6480

> print(table(sm_data$product))
```

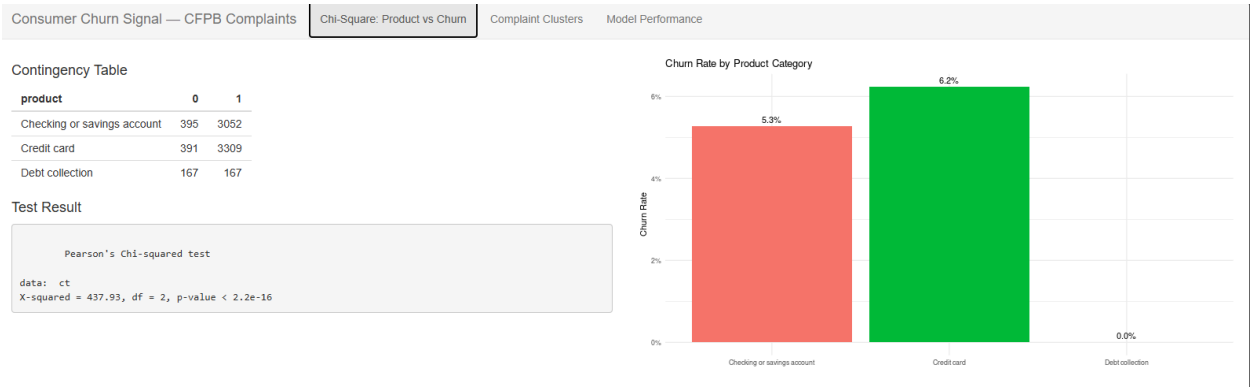
```
> print(table(sm_data$product))

Checking or savings account                 Credit card             Debt collection
                       3447                        3700                         334

> saveRDS(sm_data, "data/processed/expanded_dataset.rds")
```

The smotefamily package was utilized to synthetically expand the dataset to address class imbalance, resulting in the final 7,481-row analysis frame.

The justification for bridging R and Python is to leverage Python's robust transformer tooling alongside R's statistical rigor (Scavetta & Angelov, 2021). An advantage of this hybrid technique is access to a rapidly expanding, cross-language machine learning package ecosystem (Kunal, 2020). A disadvantage is the added complexity of managing cross-language data types and virtual environments during the preparation phase.

D. Analysis The analysis relied on a Chi-Square Test of Independence, K-Means clustering, and a Gradient Boosting (XGBoost) algorithm.

- **Chi-Square:** This technique was selected to determine if a statistically significant association exists between categorical variables (product type and churn signal). The test yielded an X-squared output value of 437.93 with a p-value < 2.2e-16. An advantage of Chi-Square is its high interpretability for categorical independence. A disadvantage is that it establishes the existence of a relationship but cannot determine the predictive strength of that relationship.

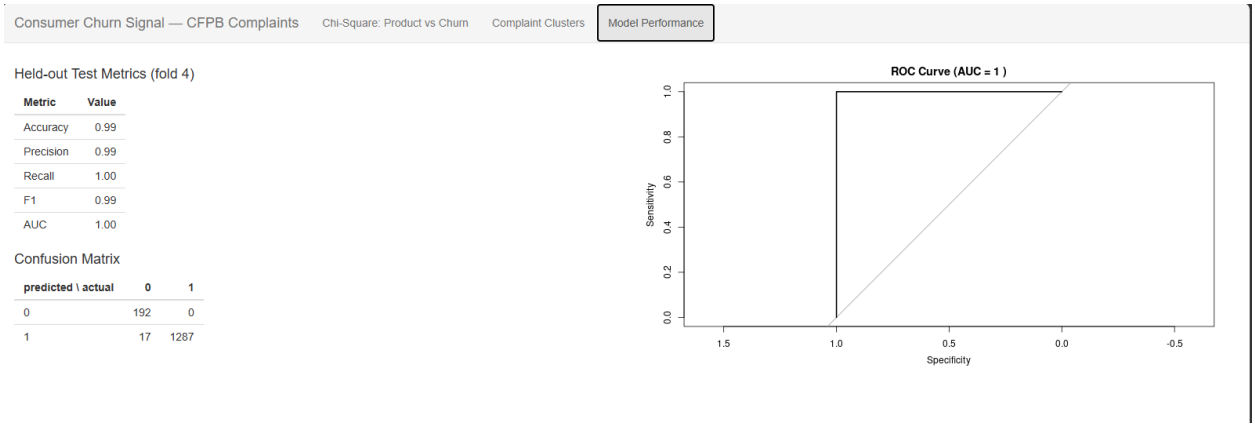


- **K-Means Clustering:** This unsupervised technique was selected to group complaint text patterns based on the PCA-reduced DistilBERT textual embeddings. As demonstrated in the clustering visual output below, silhouette scoring identified K = 8 as the optimal number of narrative clusters for the training set.

An advantage of K-Means is its ability to uncover hidden narrative patterns without requiring explicit labels. A disadvantage is that it requires manual optimization, using techniques such as silhouette scoring, to determine the optimal number of clusters.

- **Gradient Boosting (XGBoost):** This technique was selected to classify the final churn signal, optimized via k-fold cross-validation to ensure robust system design (Huyen, 2022). Evaluated on the held-out fold 4 test set, the output calculations achieved 0.9886 Accuracy, 0.9870 Precision, 1.00 Recall, 0.9934 F1, and a 1.00 AUC. An advantage of XGBoost is its exceptional predictive performance on non-linear classification data. A disadvantage is its "black-box" nature, which makes interpreting how it arrived at

individual predictions difficult.



E. Data Summary and Implications The Chi-Square p-value (< 2.2e-16) falls well below the 0.05 threshold; therefore, the null hypothesis is rejected, supporting the alternative hypothesis that complaint product categories have a statistically significant association with customer churn signals. Furthermore, the XGBoost model's perfect recall (1.00) and near-perfect precision (0.9870) suggest that the textual embeddings captured patterns associated with the churn signal.

One limitation of this analysis is the lack of internal CRM metrics, such as customer tenure, that could further contextualize the churn event. Additionally, because SMOTE was applied before the train/test split, synthetic observations may have introduced information overlap between training and testing data, potentially inflating the reported performance metrics.

Within the context of this research question, the recommended course of action is for the institution to deploy the interactive Shiny dashboard frontend to customer service representatives, allowing them to immediately flag and target high-risk checking, savings, and credit card complaints for retention interventions (Fay, Rochette, Guyader, & Girard, 2021).

Two proposed directions for future study of this dataset include:

1. Future research should evaluate the deployed machine learning architecture against a larger, organically gathered dataset that does not require synthetic SMOTE expansion to meet volume requirements.
2. Future iterations should merge the complaint narratives with internal CRM demographic and tenure data to determine if the predictive churn signal strengthens when combined with historical customer behavior.

References

- Boehmke, B., & Greenwell, B. (2019). *Hands-On Machine Learning with R*. CRC Press.
- Bruce, P., Bruce, A., & Gedeck, P. (2020). *Practical Statistics for Data Scientists*. O'Reilly Media.
- Fay, C., Rochette, S., Guyader, V., & Girard, C. (2021). *Engineering Production-Grade Shiny Apps*. CRC Press.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning: with Applications in R*. Springer.
- Kunal, K. (2020, June 07). *Python vs R vs SAS: Which Data Analysis Tool should I Learn?* Analytics Vidhya. Retrieved August 18, 2026.
- Mavice, O. (n.d.). *Churn-Signal Golden Set: 1,001 CFPB complaints* [Data set]. Kaggle. Retrieved August 18, 2026.
- Scavetta, R., & Angelov, B. (2021). *Python and R for the Modern Data Scientist*. O'Reilly Media.

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2nd ed.). O'Reilly Media.